

From Model-Centric to System-Aware Machine Learning Systems



·Anders Launer Bæk-Petersen·Jakob Hviid·
·Mikkel Baun Kjærgaard·Mahyar Turchi Moghaddam·

SDU Software Engineering

Background and Motivation

This work is **motivated** by a simple yet critical question: How can ML practitioners think beyond Machine Learning (ML) performance and design Machine Learning Systems (MLSystems) that are robust, discoverable, and observable in real-world deployments?

The motivation is rooted in previous work [1], which identified knowledge gaps centered around reliable MLSystems. The work presents a simplified view of a MLSystem (Fig. 2), and argues how a dynamic stimulus, illustrated in Fig. 1, influences the ML artifact and downstream consumer(s). Previous research by [2] highlights a significant absence of real-time feedback loops, leading to unnoticed events such as silent failures. They contend that ML performance metrics are determined offline during development using ground truth labeled datasets. However, in production environments, these ground truth labels are often delayed or unavailable, complicating the evaluation of an ML model's real-time performance. Here, conventional observability tools, which focus primarily on infrastructure metrics, often fall short.

System-aware MLSystems acknowledge that ML models are integrated within intricate, evolving software environments rather than being standalone entities. It utilizes observability to connect model performance with real-world system state in production environments. However, many ML practitioners still rely solely on model-specific metrics, neglecting the broader behavior of the MLSystem.

The expected **contribution** is to enable MLSystems to be observable continuously and adjusted according to the state of the system in real-time. This focus minimizes silent failures and unforeseen degradations, facilitating feedback loops that initiate retraining, rollback, or reconfiguration based on observed system state in MLSystems.

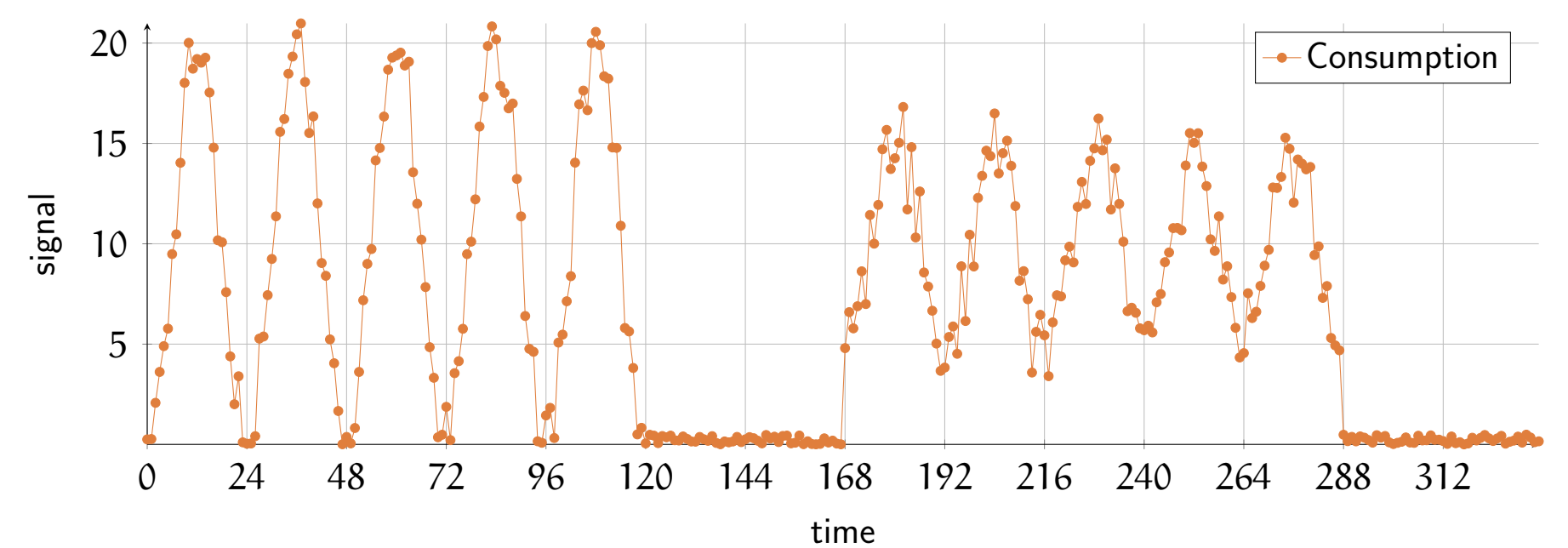


Figure 1: A synthetic upstream source that changes dynamics over time.

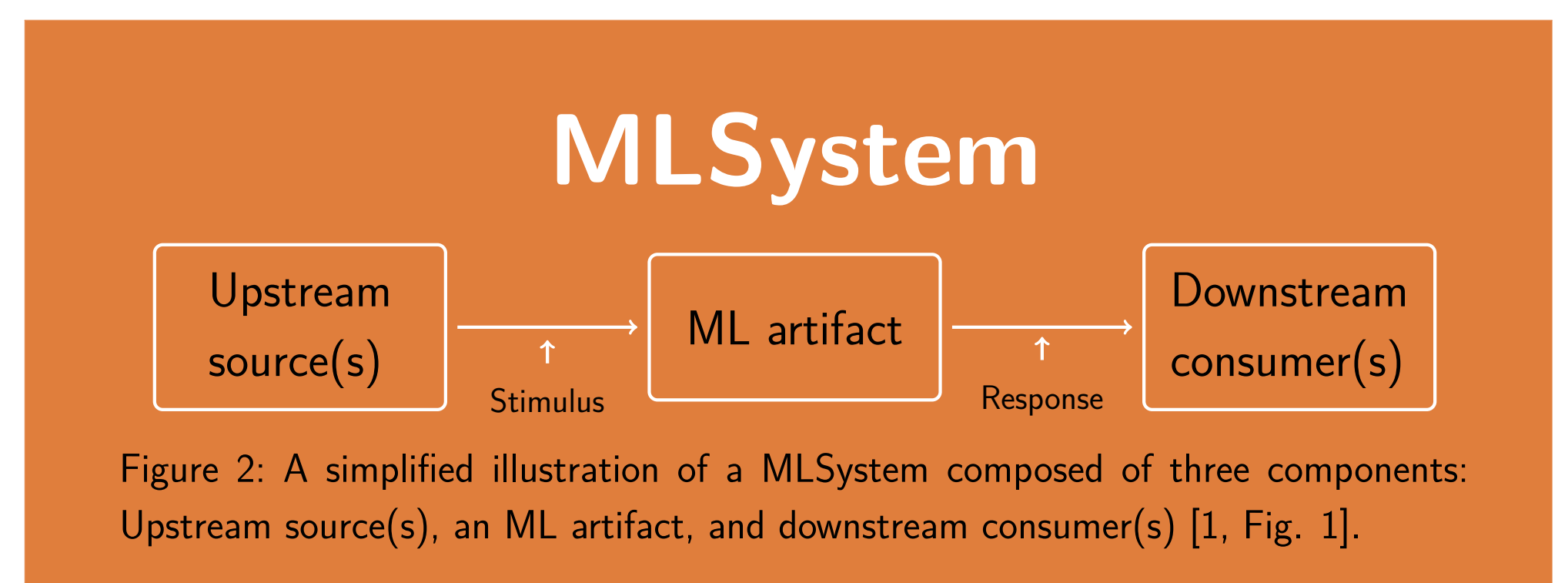


Figure 2: A simplified illustration of a MLSystem composed of three components: Upstream source(s), an ML artifact, and downstream consumer(s) [1, Fig. 1].

You built a great ML model ...
... But you need to deploy a System-aware MLSystem!

Machine Learning Performance

In ML processes, performance is primarily assessed during model development using one of many established evaluation metrics. These metrics are derived by comparing the model's predictions to offline ground truth labels from a separate validation or test dataset. Typical metrics include: accuracy, precision & recall, F1-score, ROC-AUC, MSE, etc. This process assumes that the evaluation data accurately reflects real-world conditions, anticipating that the model's performance on this data will translate to its efficacy in production. However, this assumption often fails in practice due to factors such as data drift, distribution shifts, and system-level constraints (e.g., latency, throughput, resource usage) [1, 2]. As a result, while these metrics are vital, they are inadequate for assessing a model's performance in a live system, as the combination of the data and type of model infers the real quality of the inference (Fig. 3) [3].

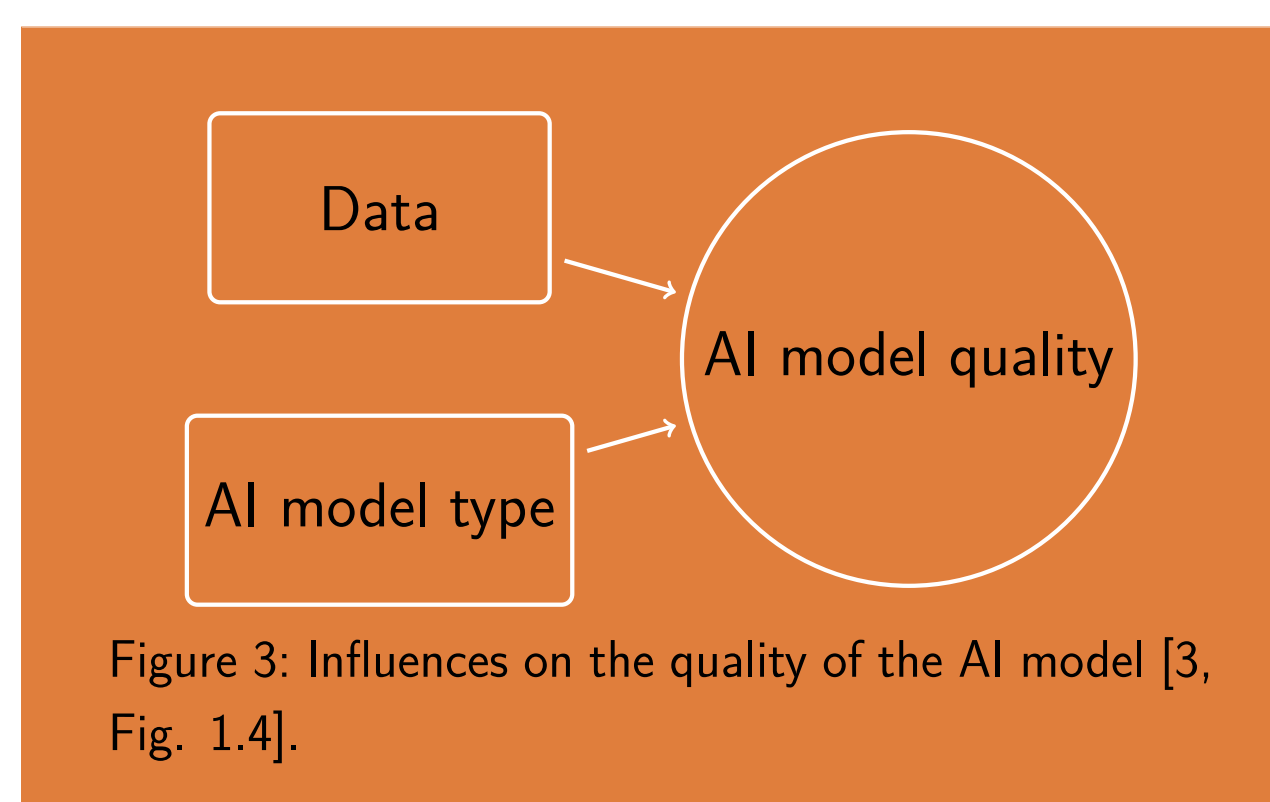


Figure 3: Influences on the quality of the AI model [3, Fig. 1.4].

The question ...
What enable your
ML model to be
System-aware?

Identified Knowledge Gap

"The gap is the absence of adequate operational quality metrics for reliable MLSystems, which include the dynamics of upstream sources, ML artifacts, and downstream consumers in operation" - [1].

Software Observability

Software Observability (SoftObs) must expand beyond infrastructure metrics to include ML-specific signals such as data drift, prediction confidence, and feedback loops to achieve visibility, diagnose issues, and adapt ML behavior in real-time.

The shift to System-aware further enables the ML practitioners to move from reactive troubleshooting to proactive system management, ensuring that deployed models remain reliable, interpretable, and aligned with real-world conditions. By incorporating relevant ML-specific signals into observability frameworks (such as Prometheus, OpenTelemetry, or customized dashboards), practitioners can detect silent failures, monitor the deterioration of ML models, and initiate reconfiguration or retraining processes.

In this way, SoftObs serves as a foundational enabler of System-aware MLSystems, bridging the gap between your great ML model and the MLSystem.

References

- [1] A. L. Bæk-Petersen, "Survey on operational metrics for reliable machine learning systems," in *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*, 2025, pp. 137–142.
- [2] S. Shankar and A. Parameswaran, "Towards Observability for Production Machine Learning Pipelines," Jul. 2022, arXiv:2108.13557 [cs]. [Online]. Available: <http://arxiv.org/abs/2108.13557>
- [3] L. Bass, Q. Lu, I. Weber, and L. Zhu, *Engineering AI systems*. Boston, MA: Addison Wesley, Mar. 2025.